Healthcare SQL Queries

Author: A. Bharathi

Setup

```
CREATE DATABASE healthcare;
USE healthcare;

-- Import CSV files into MySQL tables.
```

Task 1 – Inner Join

```
SELECT
    a.appointment_id,
    p.name AS patient_name,
    a.appointment_date,
    a.reason
FROM appointments AS a
INNER JOIN patients AS p
    ON a.patient_id = p.patient_id;
```

Task 2 – Left Join with NULL Handling

```
SELECT
    p.patient_id,
    p.name,
    p.age,
    p.gender,
    p.address
FROM patients AS p
LEFT JOIN appointments AS a
    ON p.patient_id = a.patient_id
WHERE a.appointment_id IS NULL;
```

Task 3 – Right Join & Aggregate

```
SELECT
    d.name AS doctor_name,
    d.specialization,
    COUNT(di.diagnosis_id) AS total_diagnoses
FROM doctors AS d
RIGHT JOIN diagnoses AS di
    ON d.doctor_id = di.doctor_id
GROUP BY d.doctor_id, d.name, d.specialization;
```

Task 4 – Full Join

```
SELECT *
FROM patients
LEFT JOIN appointments
    ON patients.patient_id = appointments.patient_id
UNION
SELECT *
FROM patients
RIGHT JOIN appointments
    ON patients.patient_id = appointments.patient_id;
```

Task 5 – Window Function

```
SELECT
    d.name AS doctor_name,
    p.name AS patient_name,
    COUNT(a.appointment_id) AS total_appointments,
    DENSE_RANK() OVER(
```

```
        PARTITION BY d.doctor_id
        ORDER BY COUNT(a.appointment_id) DESC
    ) AS Rankk
FROM appointments AS a
JOIN patients AS p
    ON a.patient_id = p.patient_id
JOIN doctors AS d
    ON a.doctor_id = d.doctor_id
GROUP BY d.name, p.name, d.doctor_id, p.patient_id;
```

Task 6 – CASE Expression

```
SELECT
    CASE
        WHEN age BETWEEN 18 AND 30 THEN '18-30'
        WHEN age BETWEEN 31 AND 50 THEN '31-50'
        ELSE '51+'
    END AS age_group,
    COUNT(*) AS total_patients
FROM patients
GROUP BY age_group;
```

Task 7 – String Function

```
SELECT UPPER(name) AS patient_name_upper
FROM patients
WHERE contact_number LIKE '%1234';
```

Task 8 – Subquery

```
SELECT *
FROM patients
WHERE patient_id IN (
    SELECT patient_id
    FROM diagnoses AS d
    JOIN medications AS m
        ON d.diagnosis_id = m.diagnosis_id
    GROUP BY d.patient_id
    HAVING COUNT(DISTINCT
        CASE
            WHEN m.medication_name != 'Insulin'
            THEN m.medication_name
        END
    ) = 0
);
```

Task 9 – Date Functions

```
SELECT
    d.diagnosis_id,
    AVG(DATEDIFF(m.end_date, m.start_date)) AS avg_duration_days
FROM medications AS m
JOIN diagnoses AS d
    ON m.diagnosis_id = d.diagnosis_id
GROUP BY d.diagnosis_id;
```

Task 10 – Complex Join

```
SELECT
    d.name AS doctor_name,
    d.specialization,
    COUNT(DISTINCT a.patient_id) AS unique_patients
FROM appointments AS a
JOIN doctors AS d
    ON a.doctor_id = d.doctor_id
GROUP BY d.name, d.specialization, d.doctor_id
ORDER BY unique_patients DESC
```

```
LIMIT 5;
```

Task 11

```
SELECT treatment,
       COUNT(*) AS occurrence_count
FROM diagnoses
GROUP BY treatment
ORDER BY occurrence_count DESC
LIMIT 5;
```

Task 12

```
SELECT
    p.name AS patient_name,
    COUNT(a.appointment_id) AS total_appointments
FROM patients AS p
JOIN appointments AS a
    ON p.patient_id = a.patient_id
GROUP BY p.name, p.patient_id
ORDER BY total_appointments DESC
LIMIT 4;
```

Task 13

```
SELECT
    DATE_FORMAT(appointment_date,'%Y-%m') AS month,
    COUNT(*) AS total_appointments
FROM appointments
GROUP BY month
ORDER BY month;
```

Task 14 – Custom Analytics

```
SELECT
    ROUND(COUNT(*) / COUNT(DISTINCT patient_id),2)
        AS avg_appointments_per_patient
FROM appointments;

SELECT
    DAYNAME(appointment_date) AS weekday,
    COUNT(*) AS total_appointments
FROM appointments
GROUP BY weekday
ORDER BY total_appointments DESC;
```

Additional Practice

```
Cars database queries, cohort retention analysis using CTEs, and VIEW creation with CASE expression are als
```